MathCAS v2

Quick start guide

Development build, November 2026

MathCAS v2 is a computer algebra system with its own small language: exact and arbitrary-precision numbers, symbolic algebra, rules and pattern matching, functions with types, modules, a compiler for numeric loops, and automatic parallel evaluation. This guide gets you from a first expression to scripts, modules, notebooks and editor support in a few pages. Everything shown here was run on the current build.

Contents

1 Getting started **1**

2 Numbers **2**

3 Algebra **2**

4 Lists and functions **3**

5 Defining things **3**
- 5.1 Values . . . 3
- 5.2 Functions: `fn` . . . 3
- 5.3 Rules and patterns: `rule` . . . 4
- 5.4 Blocks, loops and `match` . . . 4
- 5.5 Rewriting . . . 4

6 Number theory **4**

7 Speed: compiled functions and parallel evaluation **5**

8 Scripts, modules and notebooks **5**
- 8.1 Scripts . . . 5
- 8.2 Modules . . . 5
- 8.3 Notebooks . . . 6

9 Editors and the kernel **6**

10 Syntax at a glance **7**

11 Not there yet **7**

1 Getting started

The distribution folder holds:

`mc-repl.exe`	the interactive REPL; also runs scripts and notebooks
`mc-kernel.exe`	the kernel for notebooks and editors (JSON lines over standard input and output)
`mc-lsp.exe`	the language server for editors
`mc-bench.exe`	the benchmark harness (cases under `bench\cases`)
`examples\`	a script, a module file and a notebook to read and run
`editors\vscode`, `mathcas-0.1.0.vsix`	the VS Code extension

Start the REPL and type an expression; the value is printed below it:

```
mc> 2^100 + 1
1267650600228229401496703205377
mc> 6/4 + 1/3
11/6
```

Input continues on the next line while brackets are open. **Ctrl+C** stops a running evaluation and keeps the session. REPL commands:

`:help`	list the commands
`:time`	toggle timing (prints `time:  ...  ms` after each result)
`:history`, `:redo [k]`	show earlier inputs, run one again
`:load file.mc`	read and run a file (a module file is reloaded cleanly)
`:modules`	loaded modules and their public names
`:cancel`, `:quit`	drop the current multi-line input, leave

2 Numbers

Integers and rationals are exact and unbounded. Arithmetic stays exact unless you ask for numbers.

```
mc> x + 2x + y*x
3*x + x*y
mc> sqrt(-4)
2im
mc> exp(1.5)
4.4816890703380645
mc> 1/0
failure(division_by_zero)
```

`n(x, digits)` evaluates to arbitrary precision. Results are *balls*: a value with a guaranteed error bound, and only correct digits are shown. A ball literal is written `value~digits`.

```
mc> n(pi, 30)
3.14159265358979323846264338327950 29
mc> n(sqrt(2), 40)
1.4142135623730950488016887242096980785696719
mc> 1.25~20 * 2
2.5000000000000000000
```

Mixing kinds gives the least exact one present (exact < ball < machine real). Errors are values too: `failure(...)` terms propagate through a computation instead of stopping the session.

3 Algebra

```
mc> (1 + x)^3 |> expand
1 + 3*x + 3*x^2 + x^3
mc> factor(x^4 - 1)
```

```
(-1 + x)*(1 + x)*(1 + x^2)
mc> series(exp(x), x, 0, 5)
1 + x + x^2/2 + x^3/6 + x^4/24 + x^5/120 + O(x^6)
mc> resultant(x^2 - 2, x - y, x)
-2 + y^2
mc> gcd(2^40 - 1, 2^60 - 1)
1048575
```

Also available: `together`, `cancel`, `coefficient`, `coefficient_list`, `degree`, `variables`, `polynomial_gcd`, `polynomial_quotient`, `normal`, `discriminant` and more. A series reads back: its printed form, with `O(x^n)`, is valid input.

4 Lists and functions

Lists are written `[a, b, c]`; indexing starts at 1, and `xs[i:j]` is a slice.

```
mc> xs = [3, 1, 4, 1, 5, 9, 2, 6]
null
mc> sort(xs)
[1, 1, 2, 3, 4, 5, 6, 9]
mc> xs[2:4]
[1, 4, 1]
mc> map(k => k^2, range(5))
[1, 4, 9, 16, 25]
mc> select(range(20), is_prime)
[2, 3, 5, 7, 11, 13, 17, 19]
```

`x => body` is a function (a lambda); `&(&1 + 1)` is the short form with slots. The pipe `a |> f(b, _)` passes `a` where the `_` stands, so a computation reads left to right:

```
mc> range(10) |> map(k => k^2, _) |> sum
385
mc> fold((a, b) => a + b, 0, range(10))
55
```

`range(n)` is $1, \ldots, n$. Further list functions include `length`, `part`, `first`, `last`, `rest`, `take`, `drop`, `reverse`, `flatten`, `transpose`, `join`, `split`, `unique`, `count`, `cases`, `nest`, `nest_list`, `fold_list` and `apply`.

5 Defining things

5.1 Values

At the top level, `x = v` gives the symbol `x` a value:

```
mc> x = 5
null
mc> x^2 + 1
26
```

5.2 Functions: `fn`

A function is a list of clauses tried in order. Parameters may be typed (`Int`, `Rational`, `Real`, `Number`, `String`, `Symbol`, ...) and guarded with `when`. A call no clause accepts gives `failure(no_match, ...)`.

```
fn fact(0) = 1
```

```
fn fact(k: Int) when k > 0 = k*fact(k - 1)
```

```
mc> fact(20)
2432902008176640000
```

5.3 Rules and patterns: `rule`

Rules rewrite terms that match a pattern. `x_` matches one argument, `xs__` one or more, `xs___` zero or more; `x_: Int` adds a type. `=` evaluates the right side once, `:=` at every use.

```
rule fib(0) = 0
rule fib(1) = 1
rule fib(k_: Int) := fib(k - 1) + fib(k - 2)
```

```
mc> fib(80)
23416728348467685
```

This naive recursion is fast because MathCAS remembers normal forms it has computed (the *memo*): each `fib(k)` is evaluated once, until a definition it depends on changes.

5.4 Blocks, loops and `match`

```
mc> { var s = 0; for k in range(100) { s = s + k }; s }
5050
mc> match 7 { 0 => "zero", n_: Int when n > 0 => "positive", _ => "other" }
"positive"
```

In a block `{ ...; ... }`, `let` binds a name and `var` makes a mutable local; `while c { ... }` and `for x in xs { ... }` loop. `if c then a else b` is an expression.

5.5 Rewriting

`rewrite(e, rules)` applies rules to every part of `e` once; with `strategy: fixed_point` it repeats until nothing changes. A rule for part of a sum or product also applies inside a longer one, and the rest of a sum matched by `r___` is itself a sum.

```
mc> rewrite(x + sin(u)^2 + cos(u)^2 + y, [sin(a_)^2 + cos(a_)^2 -> 1])
1 + x + y
mc> rewrite(a + b + c, [x_ + r__ -> f(r)])
f(b + c)
mc> rewrite(f(f(f(f(x)))), [f(f(a_)) -> g(a)], strategy: fixed_point)
g(g(x))
```

6 Number theory

```
mc> factor_integer(2^32 + 1)
[[641, 1], [6700417, 1]]
mc> prime(1000)
7919
```

Also: `is_prime`, `next_prime`, `prime_pi`, `mod`, `power_mod`, `euler_phi`, `divisors`, `divisor_sigma`, `moebius_mu`, `jacobi_symbol`, `primitive_root`, `multiplicative_order`, `extended_gcd`, `lcm`, `binomial`, `factorial`, `integer_digits`, `from_digits`, `integer_length`.

7 Speed: compiled functions and parallel evaluation

Compiling. A function with typed parameters (`Int` or `Real`) whose body uses arithmetic, comparisons, `if`, `let`/`var`, `while`, `for` over ranges and the elementary functions can be compiled to machine code: put `@compiled` before `fn`, or call `compile(f)`.

```
@compiled fn collatz(k: Int) = {
  var c = k
  var s = 0
  while c != 1 { c = if mod(c, 2) == 0 then c/2 else 3c + 1; s = s + 1 }
  s
}
```

```
mc> collatz(27)
111
```

Compiled code is exact: on an integer overflow or anything it cannot do natively, the call falls back to the interpreter, so results never change. `compiled_info(f)` shows what was compiled (types, code size, how many calls ran compiled and how many fell back). A compiled call is interrupted by Ctrl+C like any other.

Parallel evaluation is on by default and needs no annotation: the arguments of a call (for example the elements of a `map`) are evaluated speculatively on up to 4 threads when they are expensive enough, and the result is always the same as a sequential run. `mc-repl --threads 1` turns it off.

8 Scripts, modules and notebooks

8.1 Scripts

A script is a `.mc` file, optionally starting with `#! mathcas 2`. `mc-repl file.mc` reads the whole file (nothing runs if it has a syntax error), runs its statements, shows what they print and the value of the last one:

```
> mc-repl examples\hello.mc
Exact arithmetic: 1/2
A big number: 2^100 = 1267650600228229401496703205376
The Collatz path of 6: [6, 3, 10, 5, 16, 8, 4, 2, 1]
Running sums: [1, 3, 6, 10, 15]
Odd numbers below 20: 10
[3, 1, 4, 5, 9, 2, 6]
```

Command line: `mc-repl [options] [file.mc ...]` with `-e SOURCE` (evaluate, repeatable), `-i` (enter the REPL afterwards), `--no-prelude`, `--threads N`. Exit codes: 0 ok, 1 read error, 2 a statement failed, 130 interrupted.

8.2 Modules

```
module temperature {
  pub fn to_kelvin(c) = c + scale      # pub: exported
  scale = 27315/100                     # private to the module
}
```

`import M` brings the public names of `M` into scope, `import M.{f, g}` only some, `import M except h` all but some; `M.f` always names a public member. Names are resolved when the source is read, never while it runs. `import vectors` loads `vectors.mc` from the folder of the importing file (or

the current folder at the REPL); a file is loaded once per session, and `:load` reloads it. The standard module `stats` is built in: `import stats` gives `mean`, `variance`, `std_dev`, `median` and `geometric_mean`.

The prelude. Every session starts with a small library written in MathCAS itself: `is_even`, `is_odd`, `clamp`, `log_base`, `count_if`, `every`, `some`, `member`, `zip`, `flat_map`, `accumulate`, `dot`, `norm`, `fix_point`, `say` (print several values on one line). `--no-prelude` starts without it.

8.3 Notebooks

A notebook (`.mcn`) is a plain text file of cells; it reads as a script too.

```
#! mathcas 2 notebook

#% text
# Prose lines start with "# ".

#% cell
print(1/3 + 1/6)
2^100
#% output
#| 1/2
#= 1267650600228229401496703205376
```

`mc-repl notebook.mcn` runs the code cells in order in one session; `mc-repl --update notebook.mcn` also writes each cell's output back below it. Output lines start with `#|` (printed), `#=` (the value) or `#!` (a failure or an error).

9 Editors and the kernel

VS Code. Install the extension with `code --install-extension mathcas-0.1.0.vsix` (or copy `editors\vscode` into your extensions folder), and keep `mc-repl.exe` and `mc-lsp.exe` together on your `PATH`. You get syntax colouring, errors as you type, go to definition (also across imports), hover, completion (`M.` lists a module's members), an outline, and the commands *MathCAS: Run File* and *MathCAS: Run Selection*.

The kernel protocol. `mc-kernel` reads one JSON request per line and answers with JSON lines, which is how notebook front ends and editors talk to it:

```
> {"type":"eval","id":1,"src":"x = 6\nx * 7"}
< {"id":1,"type":"value","printed":"42"}
< {"id":1,"type":"done","status":"ok","millis":0}
```

Requests: `eval`, `interrupt`, `complete`, `modules`, `reset`, `shutdown`.

10 Syntax at a glance

Write	Meaning
`a + b`, `a - b`, `a*b`, `a/b`, `a^b`, `2x`	arithmetic (juxtaposition multiplies)
`==`, `!=`, `<`, `<=`, `>`, `>=`	comparisons
`===`, `!==`	structurally identical / not identical
`and`, `or`, `not`, `true`, `false`, `null`	logic and constants
`[a, b]`, `xs[i]`, `xs[i:j]`	list, element, slice
`x => e`, `&(&1 + 1)`	function, short function
`a |> f(b, _)`	pipe: `f(b, a)`
`rewrite(e, rs, strategy: fixed_point)`	named option
`x = v`	define a value
`fn f(x: Int) when c = e`	function clause, typed and guarded
`rule f(x_) := e`, `rule f(0) = 1`	rule (delayed / immediate)
`x_`, `xs__`, `xs___`, `x_: Int`	patterns: one, one or more, zero or more, typed
`{ let a = 1; var b = 2; ... }`	block with an immutable and a mutable local
`if c then a else b`	conditional expression
`while c { ... }`, `for k in xs { ... }`	loops
`match e { p => r, ... }`	pattern match
`@listable`, `@compiled`, `@commutative`, ...	decorators on definitions
`module M { pub ... }`, `import M`	modules
`1.25~40`, `3im`, `pi`, `e`, `inf`	ball literal, complex unit, constants
`# ...`	comment

11 Not there yet

This is an early build. Missing so far: plots and graphics; differentiation (the prime notation `f'` is read, but not evaluated yet), equation solving, integration and differential equations; a notebook window (notebooks run from the command line and the kernel protocol is in place).